CHAPTER-7: DRL, DCL, TCL, JOINS AND SUB-QUERIES

7.1 Objective

- The objective of the document is to discuss on the various commands in Data Retrieval Language (DRL), Data Control Language (DCL), Transaction Control Language (TCL) parts of SQL.
- The document provides a detailed overview on various types of joins as well as sub-queries.

7.2 Course Content

7.2.1 DRL -Data Retrieval Language

The Data Retrieval Language consists of Select command and its clauses.

The most commonly used SQL command is SELECT statement. The SQL SELECT statement is used to query or retrieve data from a table in the database. The query may retrieve information from specified columns or from all of the columns in the table. To create a simple SQL SELECT Statement, you must specify the column(s) name and the table name. The whole query is called SQL SELECT Statement. The result is stored in a result table called result-set.

Syntax of SQL SELECT Statement:

```
SELECT column_list
FROM table-name
[WHERE Clause]
[GROUPBY clause]
[HAVING clause]
[ORDER BY clause];
```

- 'table-name' is the name of the table from which the information is retrieved.
- 'column_list' includes one or more columns from which data is retrieved.
- The code within the brackets is optional.

For example, consider the table student_details:

To select the first name of all the students the query will be:

```
SELECT First_Name FROM Student_Details;
```

	FIRST_NAME
	VARCHAR2 (20)
1	John
2	Mario
3	Tom
4	Max
5	Martha
6	Shane
7	Glenn

NOTE:
The commands are not case sensitive. The above SELECT statement can also be written as "select first_name from students_details;"

Try it yourself and verify the output:

You can also retrieve data from more than one column. For example, to select first name and last name of all the students.

```
SELECT first_name, last_name FROM student_details;
```

	FIRST_NAME	LAST_NAME
	VARCHAR2 (20)	VARCHAR2 (20)
1	John	Bravo
2	Mario	Cruise
3	Tom	Cruise
4	Max	Payne
5	Martha	Peterson
6	Shane	Watson
7	Glenn	Maxwell

NOTE:
In a SQL SELECT statement only SELECT and FROM statements are mandatory. Other clauses like WHERE, ORDER BY, GROUP BY, HAVING are optional.

To retrieve data from more than all the columns, we use '*' :

```
SELECT * FROM Student_Details:
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,004	Max	Payne	22	CPP
5	1,005	Martha	Peterson	21	CPP
6	1,006	Shane	Watson	24	Unix
7	1,007	Glenn	Maxwell	23	Unix

Now, to test your understanding, write a SQL query to display ID, First Name and Age from Student_Details table.

Use of expressions in SQL Select Statement

Expressions combine many arithmetic operators, they can be used in SELECT, WHERE and ORDER BY Clauses of the SQL SELECT Statement.

The operators are evaluated in a specific order of precedence, when more than one arithmetic operator is used in an expression. The order of evaluation is: parentheses, division, multiplication, addition, and subtraction. The evaluation is performed from the left to the right of the expression.

For example: If we want to display the first and last name of students combined together, we need to use the concat operator.

The SQL Select Statement would be like:

```
SELECT first_name ||' '|| last_name FROM STUDENT_DETAILS;
```

	FIRST_NAME||''||LAST_NAME
	VARCHAR2 (41)
1	John Bravo
2	Mario Cruise
3	Tom Cruise
4	Max Payne
5	Martha Peterson
6	Shane Watson
7	Glenn Maxwell

Also, an alias name can be provided using **'AS'**

```
SELECT first_name ||' '|| last_name AS Name FROM STUDENT_DETAILS;
```

	NAME
	VARCHAR2 (41)
1	John Bravo
2	Mario Cruise
3	Tom Cruise
4	Max Payne
5	Martha Peterson
6	Shane Watson
7	Glenn Maxwell

This can be called as alias for column. Similarly, we have alias for tables.

Try it yourself and verify the output:

```
SELECT s.first_name FROM student_details s;
```

In the above query, alias 's' is defined for the table student_details and the column first_name is selected from the table.

Aliases is more useful when

- ⇧ There are more than one tables involved in a query,
- ⇧ Functions are used in the query,
- ⇧ The column names are big or not readable,
- ⇧ More than one columns are combined together

7.2.2 SQL WHERE Clause

The WHERE Clause is used when we want to retrieve specific information from a table excluding other irrelevant data.
For example, when we want to see the information about students with Branch as JAVA then information of students in other branch becomes irrelevant. Retrieving information about all the students would increase the processing time for the query.

So SQL offers a feature called WHERE clause, which we can use to restrict the data that is retrieved. The condition provided in the WHERE clause filters the rows retrieved from the table and gives only those rows which are expected.
WHERE clause can be used along with SELECT, DELETE, UPDATE statements.

Syntax of SQL WHERE Clause:

```
WHERE {column or expression} comparison-operator value
```

Syntax for a WHERE clause with Select statement is:

```
SELECT column_list FROM table-name WHERE condition;
```

column or expression - Is the column of a table or a expression
comparison-operator - operators like = < > etc.
value - Any user value or a column name for comparison

Try it yourself and verify the output:

To find the name of a student with branch as "JAVA", the query would be like:

```
SELECT * FROM STUDENT_DETAILS WHERE BRANCH='Java';
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java

Comparison Operators and Logical Operators are used in WHERE Clause. These operators are discussed in the later sections.

NOTE: Aliases defined for the columns in the SELECT statement cannot be used in the WHERE clause to set conditions. Only aliases created for tables can be used to reference the columns in the table.

Now, to test your understanding, write a SQL query to display ID, First Name and Age from Student_Details table where age is 22.

How to use expressions in the WHERE Clause?

Expressions can also be used in the WHERE clause of the SELECT statement.

For example: Lets consider the Student_Details table. If we want to display the details of students whose age is less than 22 then the query will be:

```
SELECT * FROM STUDENT_DETAILS WHERE AGE<22;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,005	Martha	Peterson	21	CPP

NOTE:
Aliases defined in the SELECT Statement can be used in WHERE Clause.

7.2.3 Comparison Operators:

Comparison operators are used to compare the column data with specific values in a condition.

Comparison Operators are also used along with the SELECT statement to filter data based on specific conditions.

The below table describes each comparison operator.

Comparison Operators	Description
=	equal to
<>, !=	is not equal to
<	less than
>	greater than
>=	greater than or equal to
<=	less than or equal to

Try it yourself and verify the output:

```
SELECT * FROM STUDENT_DETAILS WHERE AGE!=22;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,005	Martha	Peterson	21	CPP
5	1,006	Shane	Watson	24	Unix
6	1,007	Glenn	Maxwell	23	Unix

To test your understanding, write a SQL query to display ID, First Name, Last Name and Age from Student_Details table where Branch is not Java.

7.2.3.1 Logical Operators:

There are three Logical Operators namely, AND, OR, and NOT. These operators compare two conditions at a time to determine whether a row can be selected for the output. When retrieving data using a SELECT statement, we can use logical operators in the WHERE clause, which allows us to combine more than one condition.

Logical Operators	Description
OR	For the row to be selected at least one of the conditions must be true.
AND	For a row to be selected all the specified conditions must be true.
NOT	For a row to be selected the specified condition must be false.

Try it yourself and verify the output:

SELECT * FROM STUDENT_DETAILS WHERE BRANCH='Java' AND AGE=21;

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,002	Mario	Cruise	21	Java
2	1,003	Tom	Cruise	21	Java

SELECT * FROM STUDENT_DETAILS WHERE BRANCH='Java' OR AGE=21;

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,005	Martha	Peterson	21	CPP

SELECT * FROM STUDENT_DETAILS WHERE NOT BRANCH='Java';

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,004	Max	Payne	22	CPP
2	1,005	Martha	Peterson	21	CPP
3	1,006	Shane	Watson	24	Unix
4	1,007	Glenn	Maxwell	23	Unix

Multiple logical operators can be used in an SQL statement. When we combine the logical operators in a SELECT statement, the order in which the statement is processed is

1) NOT
2) AND
3) OR

Try it yourself and verify the output:

```
SELECT * FROM student_details WHERE age >= 20 AND age <= 22  OR NOT branch =
'UNIX'
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,004	Max	Payne	22	CPP
5	1,005	Martha	Peterson	21	CPP
6	1,006	Shane	Watson	24	Unix
7	1,007	Glenn	Maxwell	23	Unix

To test your understanding, write a SQL query to display ID, First Name, Last Name and Age from Student_Details table where Branch is not Java or age is greater than 21 and less than 23.

7.2.4 SQL Comparison Keywords

The comparison keywords available in sql are used to enhance the search capabilities of a sql query. They are "IN", "BETWEEN...AND", "IS NULL", "LIKE".

Comparision Operators	Description
LIKE	column value is similar to specified character(s).
IN	column value is equal to any one of a specified set of values.
BETWEEN...AND	column value is between two values, including the end values specified in the range.
IS NULL	column value does not exist.

7.2.4.1 SQL LIKE Operator

The LIKE operator is used to display all rows in a table whose column values match a specified pattern. It is useful when we want to search rows to match a specific pattern, or when we do not know the entire value. For this purpose we use wildcard characters '%' (Percentage) and '_' (Underscore).

%	A substitute for zero or more characters
_	A substitute for a single character

Try it yourself and verify the output:

If we want to fetch the details of student whose name starts with 'J', the query can be as follows:

```
SELECT * FROM STUDENT_DETAILS WHERE FIRST_NAME LIKE 'J%';
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java

To fetch the records where we know some pattern of a field, the query can be as follows:

```
SELECT * FROM STUDENT_DETAILS WHERE FIRST_NAME LIKE '% a %' ;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,002	Mario	Cruise	21	Java
2	1,004	Max	Payne	22	CPP
3	1,005	Martha	Peterson	21	CPP
4	1,006	Shane	Watson	24	Unix

Note:
The data inside quotes is case sensitive. The following query will not fetch any data since 'A' is in Upper Case:

```
SELECT * FROM STUDENT_DETAILS WHERE FIRST_NAME LIKE '% A %' ;
```

Hence no records are fetched.

To match a one character, we use '_' as follows:

```
SELECT * FROM STUDENT_DETAILS WHERE FIRST_NAME LIKE '_om%';
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,003	Tom	Cruise	21	Java

Hence, it matches 'T'om and displays that.

NOTE:
Each underscore acts as a placeholder for only one character. So we can use more than one underscore.

Try it yourself and verify the output:

```
SELECT * FROM STUDENT_DETAILS WHERE FIRST_NAME LIKE 'M_r__a';
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,005	Martha	Peterson	21	CPP

Now, to test your understanding, write a SQL query to display ID, First Name, Last Name and Age from Student_Details table where Last Name starts with 'C' and ends with 'e'.

7.2.4.1 SQL BETWEEN ... AND Operator

The operator BETWEEN and AND, are used to compare data for a range of values.
For Example:

```
SELECT * FROM STUDENT_DETAILS WHERE AGE BETWEEN 20 AND 22;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,004	Max	Payne	22	CPP
5	1,005	Martha	Peterson	21	CPP

NOTE:

The BETWEEN...AND operator also includes the first and last values along with the middle values as in the given example, 20 and 22 are also included.

7.2.4.1 SQL IN Operator

The IN operator is used to compare a column with more than one value. It is similar to an OR condition.

Try it yourself and verify the output:

To display the details of students with Branch as 'Java' or 'Unix', the query can be:

```
SELECT * FROM STUDENT_DETAILS WHERE BRANCH IN('Java','Unix') ;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,006	Shane	Watson	24	Unix
5	1,007	Glenn	Maxwell	23	Unix

NOTE:
The data used to compare is case sensitive.

7.2.4.1 SQL IS NULL Operator

A column value is NULL if it does not exist. The IS NULL operator is used to display all the rows for columns that do not have a value.

Try it yourself and verify the output:

If we want to display the details of student who has not registered for any course then the query will be:

```
SELECT * FROM STUDENT_DETAILS WHERE BRANCH IS NULL;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,008	Brett	Lee	26	NULL

There would be no output if every student has registered in a course in the table student_details, else the names of the students who have not registered would be displayed.

7.2.5 SQL Order By

The ORDER BY clause is used in a SELECT statement to sort results either in ascending or descending order. Oracle sorts query results in ascending order by default.

Syntax for using SQL ORDER BY clause to sort data is:

SELECT column-list
FROM table_name [WHERE condition]
[ORDER BY column1 [, column 2, .. [column N]] [ASC | DESC]];

The database records are as follows:

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,004	Max	Payne	22	CPP
5	1,005	Martha	Peterson	21	CPP
6	1,006	Shane	Watson	24	Unix
7	1,007	Glenn	Maxwell	23	Unix

Try it yourself and verify the output:

To fetch the records of students in ascending order of the age then the query will be:

```
SELECT * FROM STUDENT_DETAILS ORDER BY AGE;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,005	Martha	Peterson	21	CPP
5	1,004	Max	Payne	22	CPP
6	1,007	Glenn	Maxwell	23	Unix
7	1,006	Shane	Watson	24	Unix

By default the data is sorted in ascending order, hence if we want to display it in descending order then the query will be:

```
SELECT * FROM STUDENT_DETAILS ORDER BY AGE DESC;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,006	Shane	Watson	24	Unix
2	1,007	Glenn	Maxwell	23	Unix
3	1,004	Max	Payne	22	CPP
4	1,002	Mario	Cruise	21	Java
5	1,005	Martha	Peterson	21	CPP
6	1,003	Tom	Cruise	21	Java
7	1,001	John	Bravo	20	Java

The data can also be sorted by multiple columns as shown in below query:

```
SELECT * FROM STUDENT_DETAILS ORDER BY AGE, ID;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)
1	1,001	John	Bravo	20	Java
2	1,002	Mario	Cruise	21	Java
3	1,003	Tom	Cruise	21	Java
4	1,005	Martha	Peterson	21	CPP
5	1,004	Max	Payne	22	CPP
6	1,007	Glenn	Maxwell	23	Unix
7	1,006	Shane	Watson	24	Unix

Hence, the data will be sorted by 'Age' first and then by 'Id'.

We can represent the columns in the ORDER BY clause by specifying the position of a column in the SELECT list, instead of writing the column name.

The above query can also be written as given below:

```
SELECT Age,ID FROM STUDENT_DETAILS ORDER BY 1,2;
```

	AGE	ID
	NUMBER (10)	NUMBER (10)
1	20	1,001
2	21	1,002
3	21	1,003
4	21	1,005
5	22	1,004
6	23	1,007
7	24	1,006

```
SELECT Age,ID FROM STUDENT_DETAILS ORDER BY 1 desc,2 desc;
```

	AGE	ID
	NUMBER (10)	NUMBER (10)
1	24	1,006
2	23	1,007
3	22	1,004
4	21	1,005
5	21	1,003
6	21	1,002
7	20	1,001

To test your understanding, write a SQL query to display ID, First Name, Last Name and Age from Student_Details table in ascending order of First Name and descending order of Last Name.

7.2.5.1 How to use expressions in the ORDER BY Clause?

Expressions in the ORDER BY clause of a SELECT statement. Suppose we have to add one year to age of all the students, and then sort in order of the new age then the query would be

Try it yourself and verify the output:

```
SELECT ID, FIRST_name, last_name,age,age+1 AS new_age FROM student_details ORDER
BY NEW_Age Desc;
```

	ID	FIRST_NAME	LAST_NAME	AGE	NEW_AGE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	NUMBER
1	1,006	Shane	Watson	24	25
2	1,007	Glenn	Maxwell	23	24
3	1,004	Max	Payne	22	23
4	1,002	Mario	Cruise	21	22
5	1,005	Martha	Peterson	21	22
6	1,003	Tom	Cruise	21	22
7	1,001	John	Bravo	20	21

7.2.5 SQL Distinct Keyword

This keyword is used to select the distinct rows.

Try it yourself and verify the output:

To select all distinct department names from employee table, the query would be

```
SELECT DISTINCT BRANCH FROM STUDENT_DETAILS;
```

	BRANCH
	VARCHAR2 (20)
1	Unix
2	Java
3	CPP

Hence, this keyword can be used to fetch the non redundant data from the table.

7.2.6 SQL Aggregate Functions

Group functions are built-in SQL functions that operate on groups of rows and return one value for the entire group. These functions are: COUNT, MAX, MIN, AVG, SUM

SQL COUNT ()
This function returns the number of rows in the table that satisfies the condition specified in the WHERE condition. If the WHERE condition is not specified, then the query returns the total number of rows in the table.

Try it yourself and verify the output:

If you want the number of students in a particular department, the query would be:

```
SELECT COUNT (*) FROM STUDENT_DETAILS
WHERE BRANCH = 'Java';
```

	COUNT(*)
	NUMBER
1	3

If you want the total number of students in all the department, the query would be

```
SELECT COUNT (*) FROM STUDENT_DETAILS;
```

	COUNT(*)
	NUMBER
1	7

SQL MAX()
This function is used to get the maximum value from a column.

To get the maximum age of a student, the query would be:

```
SELECT MAX (AGE) FROM STUDENT_DETAILS;
```

	MAX(AGE)
	NUMBER
1	24

SQL MIN()
This function is used to get the minimum value from a column.

To get the minimum age of a student, the query would be:

```
SELECT MIN (AGE) FROM STUDENT_DETAILS;
```

	MIN(AGE)
	NUMBER
1	20

SQL AVG()
This function is used to get the average value of a numeric column.

To get the average age of students, the query would be

```
SELECT AVG (AGE) FROM STUDENT_DETAILS;
```

	MIN(AGE)
	NUMBER
1	20

SQL SUM()
This function is used to get the sum of a numeric column

To get the total age of students,

```
SELECT SUM (AGE) FROM STUDENT_DETAILS;
```

	SUM(AGE)
	NUMBER
1	152

7.2.6 SQL Group By

The GROUP BY statement is used in conjunction with the aggregate functions to group the result-set by one or more columns.

SQL GROUP BY Syntax:

```
SELECT column_name, aggregate_function(column_name)
        FROM table_name
        WHERE column_name operator value
        GROUP BY column_name;
```

To understand the group by clause, we have added one more column as fees into the table Student_details as follows:

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,001	John	Bravo	20	Java	1,500
2	1,002	Mario	Cruise	21	Java	1,500
3	1,003	Tom	Cruise	25	Java	1,500
4	1,004	Max	Payne	22	CPP	2,500
5	1,005	Martha	Peterson	21	CPP	2,500
6	1,006	Shane	Watson	24	Unix	2,000
7	1,007	Glenn	Maxwell	23	Unix	2,000

Now if we want to display the total fees of all the branches then, the query would be

```
SELECT Branch, SUM (FEE) FROM STUDENT_DETAILS
GROUP BY Branch;
```

	BRANCH	SUM(FEE)
	VARCHAR2 (20)	NUMBER
1	Unix	4,000
2	Java	4,500
3	CPP	5,000

The expressions must be also included in the group by clause else it will not execute. Also there must be at least one table included in the from clause. Where clause can be used before group by if there are any condition/s to be met.

Also remember the sequence that we discussed in the beginning of DRL that needs to be followed.

```
SELECT column_list
FROM table-name
[WHERE Clause]
[GROUPBY clause]
[HAVING clause]
[ORDER BY clause];
```

From clause is mandatory and rest are optional

7.2.7 SQL Having Clause

The SQL HAVING Clause is used in combination with the GROUP BY Clause to restrict the groups of returned rows to only those whose the condition is TRUE.
The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

SQL HAVING Syntax:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name
HAVING aggregate_function(column_name) operator value;
```

Try it yourself and verify the output:

Now if we want to display the total fees of all the branches where the fee is greater than 4000 then, the query would be

```
SELECT Branch, SUM (FEE)
FROM STUDENT_DETAILS
GROUP BY Branch
HAVING SUM(FEE)>4000;
```

	BRANCH	SUM(FEE)
	VARCHAR2 (20)	NUMBER
1	Java	4,500
2	CPP	5,000

Hence, 'Having clause' does the work of a 'where clause' for Grouping functions as shown in the example as only the fee less than 4000 is displayed.

Also, we can give the alias for the aggregate function in the following manner:

```
SELECT Branch, SUM (FEE) AS Total_Fees
FROM STUDENT_DETAILS
GROUP BY Branch
HAVING SUM(FEE)>4000;
```

	BRANCH	TOTAL_FEES
	VARCHAR2 (20)	NUMBER
1	Java	4,500
2	CPP	5,000

When WHERE, GROUP BY and HAVING clauses are used together in a SELECT statement, the WHERE clause is processed first, then the rows that are returned after the WHERE clause is executed are grouped based on the GROUP BY clause.

Finally, any conditions on the group functions in the HAVING clause are applied to the grouped rows before the final output is displayed.

7.2.7 SQL Joins

The SQL Joins clause is used to combine records from two or more tables in a database by using values common to each.

SQL Join Types:
There are different types of joins available in SQL:

- ⇧ INNER JOIN: returns rows when there is a match in both tables.

- ⇧ LEFT JOIN: returns all rows from the left table, even if there are no matches in the right table.
- ⇧ RIGHT JOIN: returns all rows from the right table, even if there are no matches in the left table.
- ⇧ FULL JOIN: returns rows when there is a match in one of the tables.
- ⇧ SELF JOIN: is used to join a table to itself as if the table were two tables, temporarily renaming at least one table in the SQL statement.

To understand joins, we will add one more table as STUDENT_JOINING as shown below. Hence we will perform all the joins operations on these tables.

	JOINING_ID	JOINING_DATE	ID
	NUMBER (10)	DATE	NUMBER (10)
1	901	1/12/2014 12:00:00 AM	1,001
2	902	1/13/2014 12:00:00 AM	1,002
3	903	2/13/2014 12:00:00 AM	1,003
4	904	2/14/2014 12:00:00 AM	1,001
5	905	3/14/2014 12:00:00 AM	1,002
6	906	4/14/2014 12:00:00 AM	1,006
7	907	4/28/2014 12:00:00 AM	1,007

INNER JOIN

The most frequently used and important of the joins is the INNER JOIN. They are also referred to as an EQUIJOIN.

The INNER JOIN creates a new result table by combining column values of two tables (table1 and table2) based upon the join-predicate. The query compares each row of table1 with each row of table2 to find all pairs of rows which satisfy the join-predicate. When the join-predicate is satisfied, column values for each matched pair of rows of A and B are combined into a result row.

The basic syntax is as follows:

```
SELECT table1.column1, table2.column2...
FROM table1
INNER JOIN table2
ON table1.common_field = table2.common_field;
```

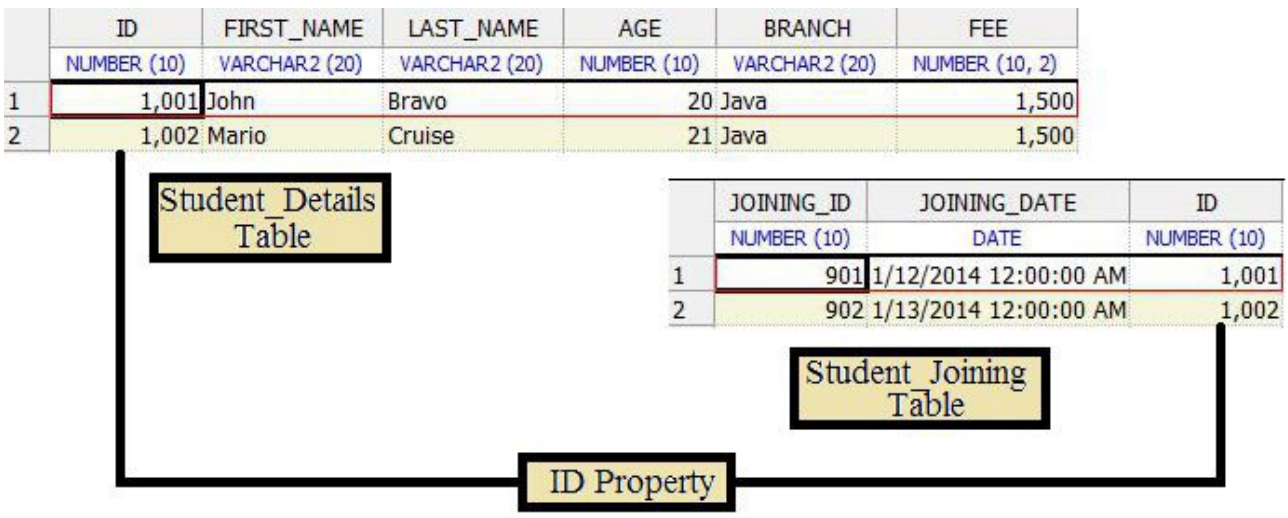
Try it yourself and verify the output:

Lets join the Student_Details and Student_Joining table and identify the joining date of each student. The query would be:

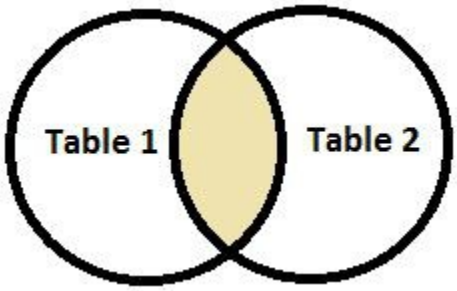
```
SELECT FIRST_NAME,LAST_NAME,FEE, JOINING_DATE
FROM STUDENT_DETAILS
 INNER JOIN STUDENT_JOINING
ON STUDENT_DETAILS.ID=STUDENT_JOINING.ID;
```

	FIRST_NAME	LAST_NAME	FEE	JOINING_DATE
	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10, 2)	DATE
1	John	Bravo	1,500	1/12/2014 12:00:00 AM
2	Mario	Cruise	1,500	1/13/2014 12:00:00 AM
3	Tom	Cruise	1,500	2/13/2014 12:00:00 AM
4	John	Bravo	1,500	2/14/2014 12:00:00 AM
5	Mario	Cruise	1,500	3/14/2014 12:00:00 AM
6	Shane	Watson	2,000	4/14/2014 12:00:00 AM
7	Glenn	Maxwell	2,000	4/28/2014 12:00:00 AM

It is also important to visualize how different joins work. Hence we will see a diagrammatic representation of all the joins.



The SQL INNER JOIN would return the records where table1 and table2 intersect.



Also, if we want to display Id that is the common field between the tables, then

```
SELECT S1.FIRST_NAME,S1.LAST_NAME,S1.FEE, S2.JOINING_DATE, S2.ID
FROM STUDENT_DETAILS S1 INNER JOIN STUDENT_JOINING S2
ON S1.ID=S2.ID;
```

	FIRST_NAME	LAST_NAME	FEE	JOINING_DATE	ID
	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10, 2)	DATE	NUMBER (10)
1	John	Bravo	1,500	1/12/2014 12:00:00 AM	1,001
2	Mario	Cruise	1,500	1/13/2014 12:00:00 AM	1,002
3	Tom	Cruise	1,500	2/13/2014 12:00:00 AM	1,003
4	John	Bravo	1,500	2/14/2014 12:00:00 AM	1,001
5	Mario	Cruise	1,500	3/14/2014 12:00:00 AM	1,002
6	Shane	Watson	2,000	4/14/2014 12:00:00 AM	1,006
7	Glenn	Maxwell	2,000	4/28/2014 12:00:00 AM	1,007

LEFT JOIN

The SQL LEFT JOIN returns all rows from the left table, even if there are no matches in the right table. This means that if the ON clause matches 0 (zero) records in right table, the join will still return a row in the result, but with NULL in each column from right table.

This means that a left join returns all the values from the left table, plus matched values from the right table or NULL in case of no matching join predicate.

The left join is also referred as left outer join

The basic syntax is as follows:

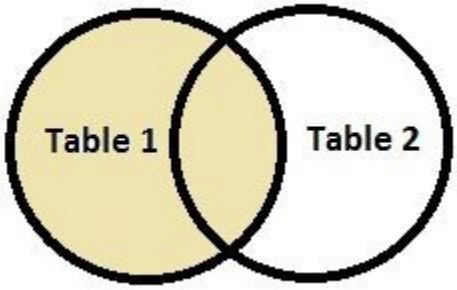
```
SELECT table1.column1, table2.column2...
FROM table1LEFT JOIN table2
ON table1.common_field = table2.common_field;
```

Try it yourself and verify the output:

```
SELECT STUDENT_DETAILS.ID, FIRST_NAME,LAST_NAME,FEE, JOINING_DATE
FROM STUDENT_DETAILS LEFT JOIN STUDENT_JOINING
ON STUDENT_DETAILS.ID=STUDENT_JOINING.ID;
```

2	1,002	Mario	Cruise	1,500	1/13/2014 12:00:00 AM
3	1,003	Tom	Cruise	1,500	2/13/2014 12:00:00 AM
4	1,001	John	Bravo	1,500	2/14/2014 12:00:00 AM
5	1,002	Mario	Cruise	1,500	3/14/2014 12:00:00 AM
6	1,006	Shane	Watson	2,000	4/14/2014 12:00:00 AM
7	1,007	Glenn	Maxwell	2,000	4/28/2014 12:00:00 AM
8	1,005	Martha	Peterson	2,500	NULL
9	1,004	Max	Payne	2,500	NULL

The diagrammatic representation of all left join:



RIGHT JOIN

The SQL RIGHT JOIN returns all rows from the right table, even if there are no matches in the left table. This means that if the ON clause matches 0 (zero) records in left table, the join will still return a row in the result, but with NULL in each column from left table.

This means that a right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.

The right join is also referred as right outer join

The basic syntax is as follows:

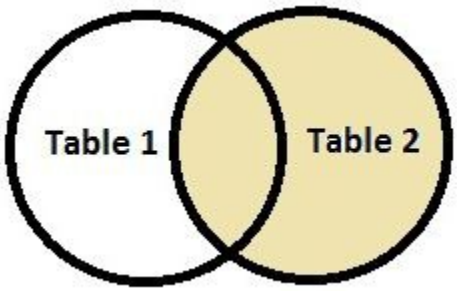
```
SELECT table1.column1, table2.column2...
FROM table1
RIGHT JOIN table2
ON table1.common_field = table2.common_field;
```

Try it yourself and verify the output:

```
SELECT STUDENT_DETAILS.ID, FIRST_NAME,LAST_NAME,FEE, JOINING_DATE
FROM STUDENT_DETAILS RIGHT JOIN STUDENT_JOINING
ON STUDENT_DETAILS.ID=STUDENT_JOINING.ID;
```

	ID	FIRST_NAME	LAST_NAME	FEE	JOINING_DATE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10, 2)	DATE
1	1,001	John	Bravo	1,500	2/14/2014 12:00:00 AM
2	1,001	John	Bravo	1,500	1/12/2014 12:00:00 AM
3	1,002	Mario	Cruise	1,500	3/14/2014 12:00:00 AM
4	1,002	Mario	Cruise	1,500	1/13/2014 12:00:00 AM
5	1,003	Tom	Cruise	1,500	2/13/2014 12:00:00 AM
6	1,006	Shane	Watson	2,000	4/14/2014 12:00:00 AM
7	1,007	Glenn	Maxwell	2,000	4/28/2014 12:00:00 AM

The diagrammatic representation of all right join:



FULL JOIN

The SQL FULL JOIN combines the results of both left and right outer joins.

The joined table will contain all records from both tables, and fill in NULL for missing matches on either side.

The basic syntax is as follows:

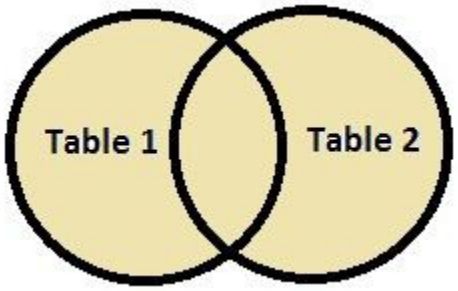
```
SELECT table1.column1, table2.column2...
FROM table1
FULL JOIN table2
ON table1.common_field = table2.common_field;
```

Try it yourself and verify the output:

```
SELECT STUDENT_DETAILS.ID, FIRST_NAME,LAST_NAME,FEE, JOINING_DATE
FROM STUDENT_DETAILS FULL JOIN STUDENT_JOINING
ON STUDENT_DETAILS.ID=STUDENT_JOINING.ID;
```

	ID	FIRST_NAME	LAST_NAME	FEE	JOINING_DATE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10, 2)	DATE
1	1,001	John	Bravo	1,500	1/12/2014 12:00:00 AM
2	1,002	Mario	Cruise	1,500	1/13/2014 12:00:00 AM
3	1,003	Tom	Cruise	1,500	2/13/2014 12:00:00 AM
4	1,001	John	Bravo	1,500	2/14/2014 12:00:00 AM
5	1,002	Mario	Cruise	1,500	3/14/2014 12:00:00 AM
6	1,006	Shane	Watson	2,000	4/14/2014 12:00:00 AM
7	1,007	Glenn	Maxwell	2,000	4/28/2014 12:00:00 AM
8	1,005	Martha	Peterson	2,500	NULL
9	1,004	Max	Payne	2,500	NULL

The diagrammatic representation of all full join:



SELF JOIN

A Self Join is a type of sql join which is used to join a table to itself, particularly when the table has a FOREIGN KEY that references its own PRIMARY KEY. It is necessary to ensure that the join statement defines an alias for both copies of the table to avoid column ambiguity.

A self join is basically when a table is joined to itself. The way you should visualize a self join for a given table is by imagining that a join is performed between two identical copies of that table. And that is exactly why it is called a self join – because of the fact that it's just the same table being joined to another copy of itself rather than being joined with a different table.

A self join must have aliases:

In a self join we are joining the same table to itself by essentially creating two copies of that table. But, how do we distinguish between the two different copies of the table – because there is only one table name after all? Well, when we do a self join, the table names absolutely must use aliases otherwise the column names would be ambiguous. In other words, we would not know which table's columns are being referenced without using aliases for the two copies of the table.

Try it yourself and verify the output:

```
SELECT  a.ID, b.FIRST_NAME, a.LAST_NAME, a.AGE, a.FEE
FROM STUDENT_DETAILS a, STUDENT_DETAILS b
WHERE a.FEE < b.FEE;
```

	ID	FIRST_NAME	LAST_NAME	AGE	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	NUMBER (10, 2)
1	1,002	Shane	Cruise	21	1,500
2	1,002	Glenn	Cruise	21	1,500
3	1,002	Martha	Cruise	21	1,500
4	1,002	Max	Cruise	21	1,500
5	1,003	Shane	Cruise	25	1,500
6	1,003	Glenn	Cruise	25	1,500
7	1,003	Martha	Cruise	25	1,500
8	1,003	Max	Cruise	25	1,500
9	1,001	Shane	Bravo	20	1,500
10	1,001	Glenn	Bravo	20	1,500
11	1,001	Martha	Bravo	20	1,500
12	1,001	Max	Bravo	20	1,500
13	1,006	Martha	Watson	24	2,000
14	1,006	Max	Watson	24	2,000
15	1,007	Martha	Maxwell	23	2,000
16	1,007	Max	Maxwell	23	2,000

7.2.8 SQL Subquery

A Subquery or Inner query or Nested query is a query within another SQL query and embedded within the WHERE clause.

A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN etc.

Certain Rules for subquery:

- ⇧ Subqueries must be enclosed within parentheses.
- ⇧ A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- ⇧ An ORDER BY cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY can be used to perform the same function as the ORDER BY in a subquery.

- ⇧ Subqueries that return more than one row can only be used with multiple value operators, such as the IN operator.

- ⇧ The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.

- ⇧ A subquery cannot be immediately enclosed in a set function.

- ⇧ The BETWEEN operator cannot be used with a subquery; however, the BETWEEN operator can be used within the subquery.

Subqueries with the SELECT Statement

Subqueries are most frequently used with the SELECT statement.
The basic syntax is as follows:

```
SELECT column_name [, column_name ]
FROM   table1 [, table2 ]
WHERE  column_name OPERATOR
(SELECT column_name [, column_name ]
FROM table1 [, table2 ]
[WHERE])
```

Try it yourself and verify the output:

Consider the Student_details table:

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,004	Max	Payne	22	CPP	2,500
2	1,005	Martha	Peterson	21	CPP	2,500
3	1,006	Shane	Watson	24	Unix	2,000
4	1,007	Glenn	Maxwell	23	Unix	2,000

Now we have learn about max() -aggregate function that it fetches the maximum value. What if we want to fetch the record with the second largest value??

Here, sub query can be used as follows:
Let us fetch the second highest ID from Student_Details table :

```
SELECT MAX(ID) FROM STUDENT_DETAILS WHERE ID NOT IN (SELECT MAX(ID)
FROM STUDENT_DETAILS);
```

	MAX(ID)
	NUMBER
1	1,006

Initially, the subquery will fetch the ID as 1007. Hence the query becomes max(ID) excluding 1007. Thus, we get the ID as 1006 that is the second highest.

Subqueries with the INSERT Statement

Subqueries also can be used with INSERT statements. The INSERT statement uses the data returned from the subquery to insert into another table. The selected data in the subquery can be modified with any of the character, date or number functions.

The basic syntax is as follows:

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

Try it yourself and verify the output:

Consider a table STUDENT_DETAILS_2 with similar structure as STUDENT_DETAILS table. Now to copy complete STUDENT_DETAILS table into STUDENT_DETAILS_2, following is the syntax:

```
INSERT INTO  STUDENT_DETAILS_2
SELECT * FROM STUDENT_DETAILS
WHERE ID IN (SELECT ID
FROM STUDENT_DETAILS) ;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,001	John	Bravo	20	Java	1,500
2	1,002	Mario	Cruise	21	Java	1,500
3	1,003	Tom	Cruise	25	Java	1,500
4	1,004	Max	Payne	22	CPP	2,500
5	1,005	Martha	Peterson	21	CPP	2,500
6	1,006	Shane	Watson	24	Unix	2,000
7	1,007	Glenn	Maxwell	23	Unix	2,000

Hence, the same details are inserted as are present in the Student_Details table.

Subqueries with the UPDATE Statement

Subquery can be used in conjunction with the UPDATE statement. Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

The basic syntax is as follows:

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

Assuming, we have STUDENT_DETAILS_2 table available which is backup of STUDENT_DETAILS table.

Try it yourself and verify the output:

Following query updates Price by 0.25 times in STUDENT_DETAILS table for all the students whose AGE is greater than or equal to 21:

```
UPDATE STUDENT_DETAILS
SET FEE = FEE* 0.25
WHERE AGE IN (SELECT AGE FROM   STUDENT_DETAILS_2
WHERE AGE >= 21 );
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,001	John	Bravo	20	Java	1,500
2	1,002	Mario	Cruise	21	Java	375
3	1,003	Tom	Cruise	25	Java	375
4	1,004	Max	Payne	22	CPP	625
5	1,005	Martha	Peterson	21	CPP	625
6	1,006	Shane	Watson	24	Unix	500
7	1,007	Glenn	Maxwell	23	Unix	500

Subqueries with the DELETE Statement

The subquery can be used in conjunction with the DELETE statement like with any other statements mentioned above.

The basic syntax is as follows:

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[ WHERE) ]
```

Try it yourself and verify the output:

Assuming, we have STUDENT_DETAILS_2 table available which is backup of STUDENT_DETAILS table.

Following example deletes records from CUSTOMERS table for all the customers whose AGE is greater than or equal to 23:

```
DELETE FROM STUDENT_DETAILS
WHERE AGE IN (SELECT AGE FROM STUDENT_DETAILS_2
WHERE AGE > 23 );
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,001	John	Bravo	20	Java	1,500
2	1,002	Mario	Cruise	21	Java	375
3	1,004	Max	Payne	22	CPP	625
4	1,005	Martha	Peterson	21	CPP	625
5	1,007	Glenn	Maxwell	23	Unix	500

7.2.2 TCL- Transaction Control Language

Transaction Control Language(TCL) commands are used to manage transactions in database.These are used to manage the changes made by DML statements. It also allows statements to be grouped together into logical transactions.

A transaction is a unit of work that is performed against a database. Transactions are units or sequences of work accomplished in a logical order, whether in a manual fashion by a user or automatically by some sort of a database program.

A transaction is the propagation of one or more changes to the database. For example, if you are creating a record or updating a record or deleting a record from the table, then you are performing transaction on the table. It is important to control transactions to ensure data integrity and to handle database errors.

Properties of Transactions:
Transactions have the following four standard properties, usually referred to by the acronym ACID:

- **Atomicity**: ensures that all operations within the work unit are completed successfully; otherwise, the transaction is aborted at the point of failure, and previous operations are rolled back to their former state.

- **Consistency**: ensures that the database properly changes states upon a successfully committed transaction.

- ⇧ **Isolation**: enables transactions to operate independently of and transparent to each other.

- ⇧ **Durability**: ensures that the result or effect of a committed transaction persists in case of a system failure.

The TCL commands are as follows:

The COMMIT Command
COMMIT command is used to permanently save any transaction into database.
The COMMIT command saves all transactions to the database since the last COMMIT or ROLLBACK command.
The syntax for commit is:

commit;

Observations of Data State when COMMIT is Pending:
Consider a scenario where you have not issued commit and also the auto commit is off. Now since we work in distributed environment, there might be multiple database client/s.
Now you performed an insert operation and never issued a commit command after that. Hence the transaction is locked.
Now some other database client tried to insert another data.
At this point of time, the query will continue to execute and execution will never be completed because it is waiting for the other client's log to get completed and hence the connection will be timed out.
The query inserted first is at the position where the second query is trying to get inserted.

Database Client 1

```
UPDATE A99CORE.STUDENT_DETAILS
SET  FIRST_NAME = 'Glenn'
     , LAST_NAME = 'Maxwell'
     , AGE = 23
     , BRANCH = 'Unix'
     , FEE = 2000
WHERE ID = 1007;
```

Script 1*

Result 1 | Messages | Query completed: UPDATE A99CORE.S

A database client updates a record without issuing commit

Database Client 2

```
UPDATE STUDENT_DETAILS
SET ID = 1007
     , FIRST_NAME = 'Glenn'
     , LAST_NAME = 'Maxnotwell'
     , AGE = 23
     , BRANCH = 'Unix'
     , FEE = 2000
WHERE ID = 1007 ;
```

Script 1*

Result 1 | Messages | Running: UPDATE STUDENT_DETAILS SET ID = 100

Another database client tries to update the same record -query will be running, execution will not be completed because it is waiting for the other client's log to get completed and connection will be timed out

The sessions at other client usually gets the data that is commit and hence the transactions are never locked.

The ROLLBACK Command

This command restores the database to last commited state. It is also use with savepoint command to jump to a savepoint in a transaction.

The syntax for rollback is:

rollback;

or

rollback to savepoint-name;

The SAVEPOINT Command

Savepoint command is used to temporarily save a transaction so that you can rollback to that point whenever necessary.

The syntax for savepoint is:

savepoint savepoint-name;

Try it yourself and verify the output:

Let us consider the Student_details table for explaining TCL

The default table is as follows:

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,001	John	Bravo	20	Java	1,500
2	1,002	Mario	Cruise	21	Java	375

Lets use some SQL queries on the above table and see the results

```
INSERT into STUDENT_DETAILS values(1003,'Rahul','Cruise',25,'CPP',2500);

commit;

UPDATE STUDENT_DETAILS set FIRST_NAME='Tom' where id=1003;

savepoint S1;

INSERT into STUDENT_DETAILS values(1004,'Max','Payne',22,'Unix',1500);

savepoint S2;

INSERT into STUDENT_DETAILS values(1005,'Martha','Peterson',25,'CPP',2500);

savepoint S3;

SELECT * from STUDENT_DETAILS;
```

The Output is as follows:

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,004	Max	Payne	22	Unix	1,500
2	1,005	Martha	Peterson	25	CPP	2,500
3	1,003	Tom	Cruise	25	CPP	2,500
4	1,001	John	Bravo	20	Java	1,500
5	1,002	Mario	Cruise	21	Java	375

Now let's execute and understand the use of rollback and savepoint:

```
ROLLBACK TO S2;
SELECT * from STUDENT_DETAILS;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,004	Max	Payne	22	Unix	1,500
2	1,003	Tom	Cruise	25	CPP	2,500
3	1,001	John	Bravo	20	Java	1,500
4	1,002	Mario	Cruise	21	Java	375

Hence, the changes made after S2 are removed from the database.

Similarly, let's execute the following query:

```
ROLLBACK TO S1;
SELECT * from STUDENT_DETAILS;
```

	ID	FIRST_NAME	LAST_NAME	AGE	BRANCH	FEE
	NUMBER (10)	VARCHAR2 (20)	VARCHAR2 (20)	NUMBER (10)	VARCHAR2 (20)	NUMBER (10, 2)
1	1,003	Tom	Cruise	25	CPP	2,500
2	1,001	John	Bravo	20	Java	1,500
3	1,002	Mario	Cruise	21	Java	375

Hence, the changes made after S1 are removed from the database.

7.2.2 DCL- Data Control Language

Data Control Language(DCL) is used to control privilege in Database. To perform any operation in the database, such as for creating tables, sequences or views we need administrative privileges.
Also, these activities are performed by Database Administrator (DBA) as it requires the knowledge of Database Architecture. Hence, as developers we need to have understanding on DCL but need not to execute the commands.

DCL commands are used to have a control on the way database schema is used or modified.
Let us consider a scenario where a project team is working to build "Employee Payroll Application".
There are 3 modules viz;

- Employee Management module (Module 1)
- PayMaster module (Module 2)
- Arrears module (Module 3)

So three teams are formed and each team takes one module

Now if the team size is big then following issues may occur:

- ⇧ Identifying who is responsible to modify (e.g Drop, Create etc.) which all tables.
- ⇧ Unintentionally some members may drop some tables, delete some important test data, resulting into loss of time and increase in effort to recover the same
- ⇧ Multiple members may change structure of database objects (like Table, View, Procedure, Function) on adhoc basis.

Hence it will result into breaking of existing code of other members. It will lead to chaos in development team.

So to address these issues, databases provide the following approach:

- ⇧ Based on logical grouping of modules, create different database schema (i.e. separate Database user per module)
- ⇧ So in above scenario, we can have 3 database users for development environment viz; "emp_dev", "pay_dev", "arrears_dev"
- ⇧ Each database user has full control on their own schema only
- ⇧ A team member is identified to be owner of this schema
- ⇧ Using database control commands (e.g. GRANT), user is restricted to perform only permissible activities.
- ⇧ For e.g. "emp_dev" will be restricted from doing create/drop tables operations in schema controlled by "pay_dev"
- ⇧ Similarly "pay_dev" and "arrears_dev" will be restricted from doing create/drop tables in schema controlled by "emp_dev"
- ⇧ Database admin user grants these privileges (viz; Create, Drop, Delete etc.) to each database user created using DCL commands
- ⇧ Any dependent scripts will be shared by each schema owner to be replicated in their database schema

Hence in this way DCL commands play a vital role in Database Management.

Privileges are of two types:

- ⇧ **System** :creating session, table etc are all types of system privilege.
- ⇧ **Object** : any command or query to work on tables comes under object privilege.

DCL defines two commands

- ⇧ **Grant** :Gives user access privileges to database.
- ⇧ **Revoke** : Take back permissions from user.

To Allow a User to create Session

grant create session to *username*;

To Allow a User to create Table

grant create table to *username*;

To provide User with some Space on Tablespace to store Table

alter user *username* quota unlimited on system;

To Grant all privilege to a User

grant sysdba to *username*;

To Grant permission to Create any Table

grant *create* any table to *username*;

To Grant permission to Drop any Table

grant *drop* any table to *username*;

To take back Permissions

revoke *create* table from *username*;